

1. Introducción

La presente memoria aborda el estudio detallado del desarrollo y análisis de algoritmos de búsqueda aplicados a diccionarios. En particular, se centrará en la determinación experimental del tiempo medio de búsqueda de elementos en un diccionario que utiliza una tabla como tipo de datos subyacente. Este enfoque permitirá explorar y comprender a fondo el rendimiento de los algoritmos en la práctica.

La investigación se centra específicamente en los algoritmos de búsqueda `linsearch`, `autolinsearch` y `binsearch`. Estos algoritmos se seleccionaron debido a su capacidad para devolver la posición de la clave buscada mediante diferentes métodos de búsqueda. A lo largo de esta memoria, se llevará a cabo un análisis exhaustivo del funcionamiento y rendimiento de estos algoritmos, tanto teórico como empírico, proporcionando una visión completa de su eficacia en situaciones prácticas de búsqueda en diccionarios basados en tablas.

2. Objetivos

Aquí indicamos el trabajo que vamos a realizar en cada apartado.

2.1 TAD diccionario

- Implementar el TAD diccionario para facilitar el análisis del rendimiento de los algoritmos de búsqueda cuya implementación también se llevará a cabo en este apartado.

2.2 Comparación del rendimiento de búsqueda

- Analizar los tiempos de ejecución de los algoritmos de búsqueda para el caso mejor, medio y peor, visualizar los datos, compararlos con la teoría y compararlos entre ellos.

3. Herramientas y metodología

Hemos desarrollado la práctica en un entorno Linux, hemos usado los editores de código VSCode y Vim, la herramienta de gestión de memoria Valgrind y el compilador GCC. Además para las gráficas y visualización de datos también hemos usado GNUPlot y Excel. En cuanto a la memoria, hemos usado el lenguaje markdown exportado a pdf.

Para todos los apartados hemos implementado control de errores y buenas prácticas de programación.

3.1 TAD diccionario

```
PDICT init_dictionary(int size, char order);
```

Implementación

- Reservamos memoria e inicializamos los valores del struct DICT y devolvemos puntero al struct creado.

```
void free_dictionary(PDICT pdict);
```

Implementación

- Liberamos el array interno del struct y liberamos el struct.

```
int insert_dictionary(PDICT dict, int key);
```

Implementacion

- Comprobamos el tamaño de la tabla, si es necesario hacemos realloc para tener más espacio.
- Si la tabla no tiene que estar ordenada introducimos la clave en **table** en la posición **n_data**.
- Si la tabla tiene que estar ordenada introducimos la clave en su posición.
- Incrementamos **n_data** y devolvemos el número de operaciones básicas (la ob es el if dentro del único for de la función).

```
int massive_insertion_dictionary(PDICT pdict, int *keys, int n_keys);
```

Implementacion

- Esta función está diseñada para insertar un gran número de claves en un diccionario. El diccionario está representado por **pdict**, que es un puntero a una estructura de diccionario. Las claves a insertar se proporcionan como un array **keys** de enteros, y **n_claves** es el número de claves en el array.
- La función comienza comprobando si **pdict**, **pdict->table**, o **keys** es NULL, o si **n_keys** es menor que 1. Si cualquiera de estas condiciones es cierta, la función devuelve inmediatamente **ERR**, indicando un error. Esta es una forma común de manejar las condiciones de error en C, donde una función no puede lanzar una excepción como en otros lenguajes.
- A continuación, la función entra en un bucle **for** que itera sobre cada clave de la matriz **keys**. Para cada clave, llama a la función **insert_dictionary**, pasándole el diccionario y la clave actual. La función **insert_dictionary** es presumiblemente responsable de insertar la clave en el diccionario y devuelve un código de estado indicando éxito o fracaso.
- Si **insert_dictionary** devuelve **ERR**, la función **massive_insertion_dictionary** devuelve inmediatamente **ERR** para indicar que se ha producido un error durante el proceso de inserción.
- Si **insert_dictionary** no devuelve **ERR**, la función añade el estado devuelto a la variable **ob**. Esto sugiere que **insert_dictionary** puede devolver un valor distinto de sólo un código de error, quizás indicando alguna información adicional sobre el proceso de inserción.
- Finalmente, una vez procesadas todas las claves, la función devuelve el valor de **ob**. Éste será la suma de todos los códigos de estado devueltos por **insert_dictionary**, a menos que se haya producido un error, en cuyo caso se habría devuelto antes **ERR**.

```
int search_dictionary(PDICT pdict, int key, int *ppos, pfunc_search method);
```

Implementación

- Esta función está diseñada para buscar una clave en un diccionario utilizando un método de búsqueda especificado. El diccionario está representado por **pdict**, que es un puntero a una estructura de diccionario. La clave a buscar se proporciona como una clave entera. La posición en la

que se encuentra la clave se almacenará en el entero apuntado por `ppos`. El método de búsqueda se proporciona como un método puntero a una función.

- La función comienza comprobando si `pdict`, `pdict->table`, `ppos` o `method` son NULL. Si alguna de estas condiciones es cierta, la función devuelve inmediatamente `ERR`, indicando un error. Esta es una forma común de manejar las condiciones de error en C, donde una función no puede lanzar una excepción como en otros lenguajes.
- Si ninguna de estas condiciones de error es verdadera, la función llama al método de búsqueda, pasando la tabla del diccionario, los índices de inicio y fin de los datos en la tabla (0 y `pdict->n_data - 1`, respectivamente), la clave a buscar y el puntero `ppos`. Se espera que el método de búsqueda busque la clave en la tabla y almacene la posición donde se encuentra en el entero apuntado por `ppos`.
- A continuación, la función devuelve el código de estado devuelto por el método de búsqueda. Esto indicará si la búsqueda tuvo éxito y, en caso afirmativo, dónde se encontró la clave. Si no se ha encontrado la clave, el método de búsqueda debe devolver un código de estado indicándolo, y la función `search_dictionary` pasará este código de estado a su invocador.

```
int bin_search(int *table, int F, int L, int key, int *ppos);
```

Implementacion

- La función toma cinco parámetros: un puntero a una tabla de matrices de enteros, dos enteros `F` y `L` que representan los índices primero y último de la sección de la matriz que se va a buscar, una clave de enteros que representa el elemento que se va a buscar y un puntero a un entero `ppos` donde se almacenará la posición del elemento encontrado.
- La función comienza comprobando si `tabla` es NULL, si `F` es menor que 0, si `L` es menor que `F`, o si `ppos` es NULL. Si alguna de estas condiciones es cierta, la función devuelve inmediatamente `ERR`, indicando un error.
- A continuación, la función entra en un bucle `while` que continúa mientras `F` sea menor o igual que `L`. Este es el bucle principal del algoritmo de búsqueda binaria. En cada iteración del bucle, calcula el índice medio `m` del intervalo de búsqueda actual e incrementa un contador `ob`.
- Si la clave del índice medio `tabla[m]` es igual a la clave de búsqueda, almacena la posición `m + 1` en `ppos` y devuelve el contador `ob`. Se utiliza `m + 1` porque la posición está basada en 1, no en 0.
- Si la clave del índice medio es mayor que la clave de búsqueda, actualiza `L` a `m - 1` para buscar en la mitad izquierda del array en la siguiente iteración. Si la clave del índice medio es menor que la clave de búsqueda, actualiza `F` a `m + 1` para buscar en la mitad derecha de la matriz en la siguiente iteración.
- Si la clave de búsqueda no se encuentra después del bucle, establece `ppos` a `NOT_FOUND` y devuelve el contador `ob`. El contador `ob` representa el número de iteraciones realizadas en la búsqueda binaria, que puede utilizarse para medir la eficiencia de la búsqueda.

```
int lin_search(int *table, int F, int L, int key, int *ppos);
```

Implementacion

- La función toma cinco parámetros: un puntero a una tabla de matrices de enteros, dos enteros `F` y `L` que representan los índices primero y último de la sección de la matriz que se va a buscar, una clave de enteros que representa el elemento que se va a buscar y un puntero a un entero `ppos` donde se almacenará la posición del elemento encontrado.
- La función comienza comprobando si `tabla` es NULL o si `L` es menor que `F`. Si alguna de estas condiciones es cierta, la función devuelve inmediatamente `ERR`, indicando un error.
- A continuación, la función entra en un bucle `while` que continúa mientras `i` sea menor o igual que `L` y el elemento actual `tabla[i]` no sea igual a la clave de búsqueda. En cada iteración del bucle, se incrementa `i`, pasando al siguiente elemento de la matriz.
- Si el bucle finaliza sin encontrar la clave de búsqueda (es decir, `i` es mayor que `L`), establece `ppos` en `NOT_FOUND`. Si se encuentra la clave de búsqueda (es decir, `tabla[i]` es igual a la clave de búsqueda), establece `ppos` en `i`.
- Por último, la función devuelve `i - F`, que representa el número de elementos comprobados durante la búsqueda. Esto puede utilizarse para medir la eficacia de la búsqueda. Tenga en cuenta que, a diferencia de la búsqueda binaria, la eficiencia de la búsqueda lineal es directamente proporcional al tamaño de la lista, ya que puede tener que comprobar cada elemento en el peor de los casos.

```
int lin_auto_search(int *table,int F,int L, int key,int *ppos);
```

Implementacion

- La función toma cinco parámetros: un puntero a una tabla de matrices de enteros, dos enteros `F` y `L` que representan los índices primero y último de la sección de la matriz que se va a buscar, una clave de enteros que representa el elemento que se va a buscar y un puntero a un entero `ppos` donde se almacenará la posición del elemento encontrado.
- La función comienza comprobando si `tabla` es NULL o si `L` es menor que `F`. Si alguna de estas condiciones es cierta, la función devuelve inmediatamente `ERR`, indicando un error.
- A continuación, la función entra en un bucle `while` que continúa mientras `i` sea menor o igual que `L` y el elemento actual `tabla[i]` no sea igual a la clave de búsqueda. En cada iteración del bucle, se incrementa `i`, pasando al siguiente elemento de la matriz.
- Si el bucle finaliza sin encontrar la clave de búsqueda (es decir, `i` es mayor que `L`), establece `ppos` en `NOT_FOUND`. Si se encuentra la clave de búsqueda (es decir, `tabla[i]` es igual a la clave de búsqueda), se comprueba si el elemento encontrado no está ya al principio de la lista (es decir, `i - 1 >= F`). Si no lo está, intercambia el elemento encontrado con el que le precede en la lista. A continuación, establece `ppos` en `i`, que es la posición original del elemento encontrado.
- Por último, la función devuelve `i - F`, que representa el número de elementos comprobados durante la búsqueda. Esto puede utilizarse para medir la eficacia de la búsqueda. Tenga en cuenta que, a diferencia de la búsqueda binaria, la eficiencia de la búsqueda lineal es directamente proporcional al tamaño de la lista, ya que puede tener que comprobar cada elemento en el peor de los casos.

3.2 Comparación del rendimiento de búsqueda

```
short average_search_time(pfunc_search metodo, pfunc_key_generator generator,
char order, int N,int n_times, PTIME_AA ptime);
```

Implementación

- La función toma seis parámetros: un puntero de función `metodo` al método de búsqueda, un puntero de función `generator` al generador de claves, un carácter `order` que indica si el diccionario está ordenado o no, un entero `N` que representa el tamaño del diccionario, un entero `n_times` que representa el número de veces que se realizará la búsqueda, y un puntero `ptime` a una estructura `TIME_AA` donde se almacenarán los resultados.
- La función comienza comprobando si `metodo`, `generator` o `ptime` son NULL, si `order` no es `SORTED` o `NOT_SORTED`, o si `N` es menor o igual que 0. Si alguna de estas condiciones es cierta, la función devuelve inmediatamente -1, indicando un error.
- A continuación, la función inicializa un diccionario `pdict` de tamaño `N` y `order`, genera una permutación de tamaño `N` e inserta la permutación en el diccionario utilizando la función `massive_insertion_dictionary`.
- A continuación, la función asigna una matriz `claves` de tamaño `n_veces * N` y genera claves utilizando la función `generator`.
- A continuación, la función entra en un bucle que se ejecuta `n_veces`. En cada iteración, mide el tiempo antes y después de la búsqueda, calcula el tiempo empleado y lo suma a `tiempo_total`. También lleva la cuenta del número total de iteraciones que ha tardado el método de búsqueda, y el número máximo y mínimo de iteraciones.
- Si se produce un error durante la búsqueda (es decir, `ppos` es `ERR` o el reloj devuelve -1), la función libera la memoria asignada, imprime un mensaje de error y devuelve `ERR`.
- Después del bucle, la función calcula el tiempo medio y el número de iteraciones dividiendo `tiempo_total` y éste entre `n_veces`, y almacena estos valores, junto con `max`, `min`, `N`, y `n_veces`, en la estructura `ptime`.
- Finalmente, la función libera la memoria asignada y devuelve `OK`.

```
short generate_search_times(pfunc_search method, pfunc_key_generator generator,
int order, char* file, int num_min, int num_max, int incr, int n_times);
```

La función toma ocho parámetros: un puntero de función `método` al método de búsqueda, un puntero de función `generator` al generador de claves, un entero `orden` que indica si el diccionario está ordenado o no, una cadena `archivo` que representa el nombre del archivo para guardar la tabla, enteros `num_min` y `num_max` que representan los tamaños mínimo y máximo del diccionario, un entero `incr` que representa el incremento de tamaño para cada medida, y un entero `n_veces` que representa el número de veces que hay que realizar la búsqueda para cada tamaño.

La función comienza comprobando si `método` o `fichero` es NULL, si `num_min` es menor que 0, si `num_max` es menor que `num_min`, o si `incr` es menor o igual que 0. Si alguna de estas condiciones es cierta, devuelve

-1.

A continuación, asigna memoria a una matriz de estructuras `TIME_AA` para almacenar las mediciones de tiempo. Si la asignación de memoria falla, devuelve -1.

A continuación, entra en un bucle que va de `num_min` a `num_max` con un incremento de `incr`. En cada iteración, imprime el tamaño actual, genera el tiempo medio de búsqueda para el tamaño actual utilizando la función `average_search_time` y almacena el resultado en la matriz `ptime`. Si `average_search_time` devuelve -1, libera la matriz `ptime` y devuelve -1.

Después del bucle, guarda la matriz `ptime` en el archivo utilizando la función `save_time_table`. Si `save_time_table` devuelve -1, libera la matriz `ptime` y devuelve -1.

Por último, libera la matriz `ptime` y devuelve 0. Esta función es un buen ejemplo de cómo medir el rendimiento de diferentes métodos de búsqueda en diferentes tamaños de datos.

Pruebas realizadas

A continuacion realizamos distintas pruebas para cada algoritmo:

4. Código fuente

TAD diccionario

```
/**
 * Initializes a dictionary with the given size and order.
 *
 * @param size The size of the dictionary.
 * @param order The order of the dictionary (SORTED or NOT_SORTED).
 * @return A pointer to the initialized dictionary, or NULL if an error
 occurred.
 */
PDICT init_dictionary(int size, char order)
{
    PDICT dictionary;

    if (size <= 0 || (order != SORTED && order != NOT_SORTED)) {
        return NULL;
    }

    dictionary = (PDICT)malloc(sizeof(DICT));

    if (dictionary == NULL) {
        return NULL;
    }

    dictionary->size = size;
    dictionary->n_data = 0;
    dictionary->order = order;
    dictionary->table = (int *)malloc(sizeof(int) * size);

    if (dictionary->table == NULL) {
        return NULL;
    }

    return dictionary;
}

/**
 * Frees the memory allocated for the dictionary.
 *
 * @param pdict A pointer to the dictionary to be freed.
 */
void free_dictionary(PDICT pdict)
{
    if (!pdict || !pdict->table) {
        return;
    }

    free(pdict->table);
    free(pdict);
}
```

```

/**
 * Inserts a key into the dictionary.
 *
 * @param pdict A pointer to the dictionary.
 * @param key The key to be inserted.
 * @return The number of comparisons made during the insertion, or ERR if
an error occurred.
 */
int insert_dictionary(PDICT pdict, int key)
{
    int i, ob = 0;
    if (!pdict || !pdict->table) {
        return ERR;
    }
    if (pdict->n_data >= pdict->size) {
        pdict->table = realloc(pdict->table, AUMENT * sizeof(pdict->table));
        if (!pdict->table) {
            return ERR;
        }
        pdict->size = AUMENT * pdict->size;
    }
    if (pdict->order == NOT_SORTED) {
        pdict->table[pdict->n_data] = key;
    } else if (pdict->order == SORTED) {
        pdict->table[pdict->n_data] = key;
        for (i = pdict->n_data - 1; i >= 0 && pdict->table[i] > key; i--) {
            ob++;
            pdict->table[i + 1] = pdict->table[i];
        }
        pdict->table[i + 1] = key;
    } else {
        return ERR;
    }
    pdict->n_data++;
    return ob;
}

/**
 * Inserts multiple keys into the dictionary.
 *
 * @param pdict A pointer to the dictionary.
 * @param keys An array of keys to be inserted.
 * @param n_keys The number of keys to be inserted.
 * @return The total number of comparisons made during the insertion, or
ERR if an error occurred.
 */
int massive_insertion_dictionary(PDICT pdict, int *keys, int n_keys)
{
    int i, ob = 0, status;
    if (!pdict || !pdict->table || !keys || n_keys < 1) {
        return ERR;
    }
    for (i = 0; i < n_keys; i++) {

```

```

        status = insert_dictionary(pdickt, keys[i]);
        if (status == ERR) {
            return ERR;
        }
        ob += status;
    }
    return ob;
}

/**
 * Searches for a key in the dictionary using the specified search method.
 *
 * @param pdickt A pointer to the dictionary.
 * @param key The key to search for.
 * @param ppos A pointer to store the position of the key if found.
 * @param method The search method to use (bin_search or lin_search).
 * @return The number of comparisons made during the search, or ERR if an
error occurred.
 */
int search_dictionary(PDICT pdickt, int key, int *ppos, pfunc_search method)
{
    if (!pdickt || !pdickt->table || !ppos || !method) {
        return ERR;
    }
    return method(pdickt->table, 0, pdickt->n_data - 1, key, ppos);
}

/**
 * Performs a binary search for a key in the table.
 *
 * @param table The table to search in.
 * @param F The first index of the search range.
 * @param L The last index of the search range.
 * @param key The key to search for.
 * @param ppos A pointer to store the position of the key if found.
 * @return The number of comparisons made during the search, or ERR if an
error occurred.
 */
int bin_search(int *table, int F, int L, int key, int *ppos)
{
    int ob = 0, m;

    if (!table || F < 0 || L < F || !ppos) {
        return ERR;
    }

    while (F <= L) {
        m = (F + L) / 2;
        ob++;

        if (table[m] == key) {
            *ppos = m + 1;
            return ob;
        } else if (key < table[m]) {

```

```

        L = m - 1;
    } else {
        F = m + 1;
    }
}
*ppos = NOT_FOUND;
return ob;
}

/**
 * Performs a linear search for a key in the table.
 *
 * @param table The table to search in.
 * @param F The first index of the search range.
 * @param L The last index of the search range.
 * @param key The key to search for.
 * @param ppos A pointer to store the position of the key if found.
 * @return The number of comparisons made during the search, or ERR if an
error occurred.
 */
int lin_search(int *table, int F, int L, int key, int *ppos)
{
    int i = F;
    if (!table || L < F) {
        return ERR;
    }
    while (i <= L && table[i] != key) {
        i++;
    }
    if (i > L) {
        *ppos = NOT_FOUND;
    } else if (table[i] == key) {
        *ppos = i;
    }
    return i - F;
}

/**
 * Performs an automatic linear search for a key in the table and moves the
key to the previous position if found.
 *
 * @param table The table to search in.
 * @param F The first index of the search range.
 * @param L The last index of the search range.
 * @param key The key to search for.
 * @param ppos A pointer to store the position of the key if found.
 * @return The number of comparisons made during the search, or ERR if an
error occurred.
 */
int lin_auto_search(int *table, int F, int L, int key, int *ppos)
{
    int i = F, aux;
    if (!table || F > L) {
        return ERR;

```

```
    }  
    while (i <= L && table[i] != key) {  
        i++;  
    }  
    if (i > L) {  
        *ppos = NOT_FOUND;  
    } else if (table[i] == key) {  
        if (i - 1 >= F) {  
            aux = table[i - 1];  
            table[i - 1] = table[i];  
            table[i] = aux;  
            *ppos = i;  
        } else {  
            *ppos = i;  
        }  
    }  
    return i - F;  
}
```

5. Resultados y Gráficas

5.1 TAD diccionario

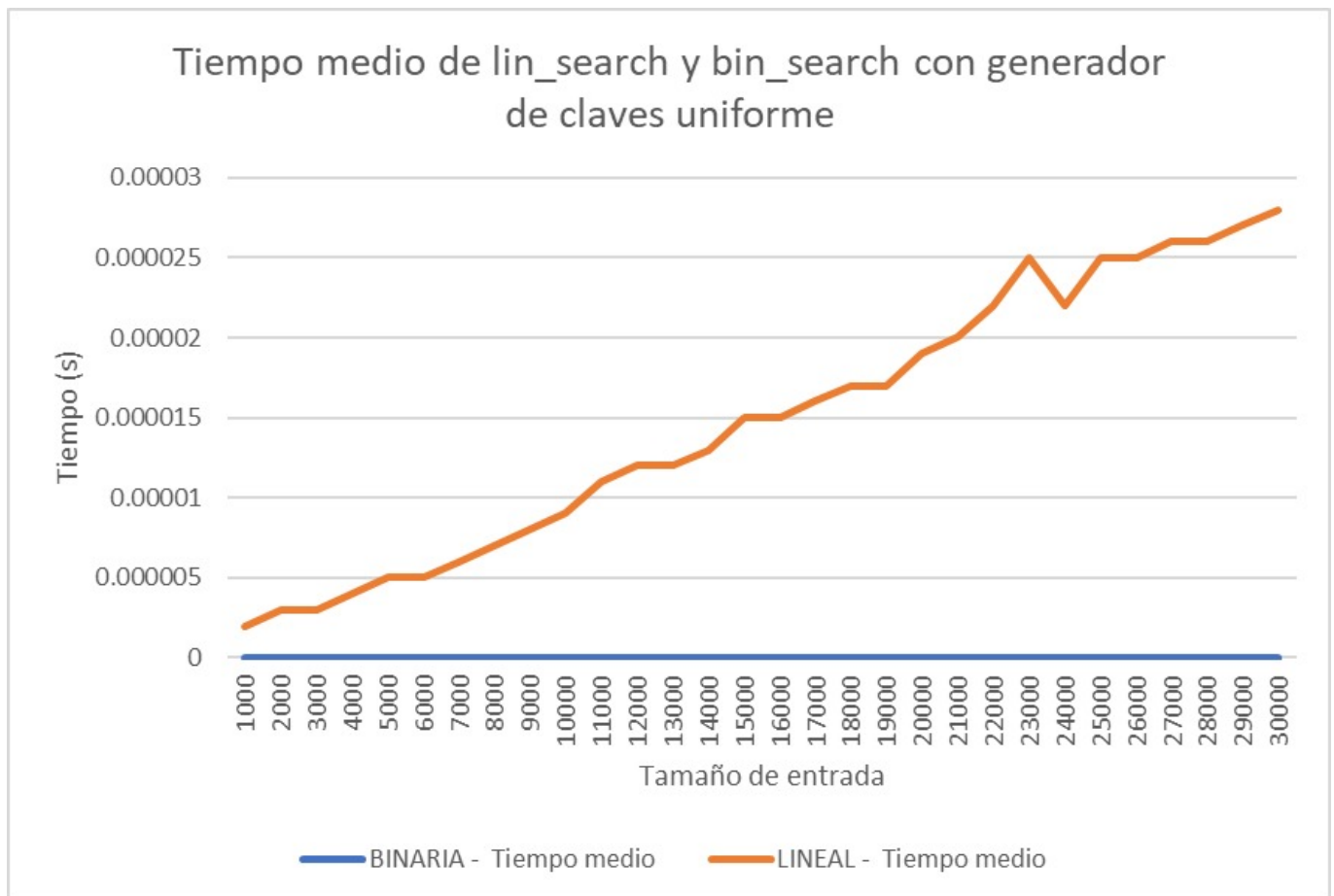
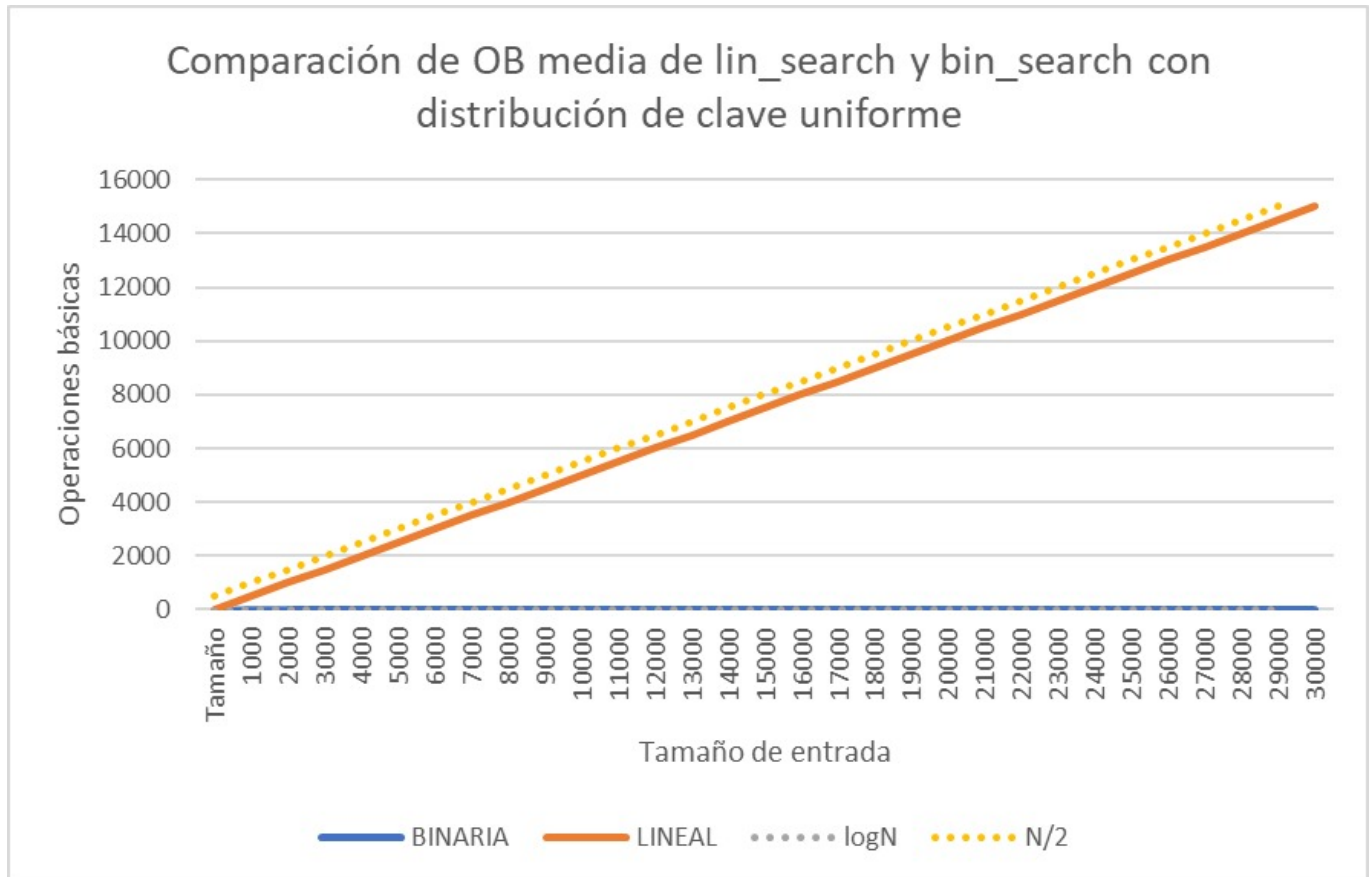
Tras ejecutar el ejercicio1 comprobamos que el TAD diccionario ha sido implementado de forma correcta tal y como explicamos en el apartado 3.1. Usar un tipo abstracto de diccionario (TAD) para realizar búsquedas y comprobar el rendimiento de algoritmos de búsqueda tiene varias ventajas:

1. **Abstracción de Datos:** Al utilizar un TAD, puedes encapsular los detalles internos de implementación del diccionario. Esto significa que el usuario de la estructura de datos no necesita conocer los detalles específicos de cómo se almacenan y gestionan los datos en el diccionario. Esto facilita la comprensión y el uso del diccionario.
2. **Interoperabilidad:** Al definir un TAD, puedes proporcionar una interfaz clara y bien definida para realizar operaciones en el diccionario. Esto facilita la interoperabilidad con otros módulos o programas, ya que los usuarios solo necesitan conocer la interfaz y no los detalles internos.
3. **Cambio de Implementación:** Si decides cambiar la implementación subyacente del diccionario (por ejemplo, cambiar de una tabla no ordenada a una tabla ordenada), los usuarios no se ven afectados siempre y cuando la interfaz del TAD se mantenga constante. Esto facilita la evolución del código y la mejora de la eficiencia sin cambiar la forma en que se utiliza el diccionario.
4. **Evaluación de Rendimiento:** Al abstraer la implementación, puedes intercambiar diferentes implementaciones del diccionario y evaluar su rendimiento sin cambiar el código que utiliza el diccionario. Esto es útil para comparar algoritmos de búsqueda, ya que puedes medir el impacto del rendimiento al cambiar la implementación subyacente.
5. **Claridad en el Código:** El uso de un TAD puede hacer que el código sea más claro y legible. Las funciones proporcionadas por el TAD actúan como operaciones bien definidas en la estructura de datos, lo que facilita la comprensión del código.

En cuanto a tu ejemplo de TAD, parece que estás implementando un diccionario con una tabla de datos, pero la implementación específica no se proporciona. Dependiendo de cómo se implementen las funciones (por ejemplo, cómo se gestiona la inserción, eliminación y búsqueda), se pueden evaluar y comparar diferentes estrategias de búsqueda y rendimiento del diccionario sin afectar el código que utiliza el TAD.

5.2 Comparación del rendimiento de búsqueda

Comando ejecutado: `./exercise2 -num_min 1000 -num_max 30000 -incr 1000 -n_times 1 -outputFile exercise5.log`



La gráfica muestra la comparación de tiempos promedio (comparaciones de elementos en el caso de búsqueda lineal y búsqueda binaria) en función del tamaño del tablero.

Para la búsqueda lineal (lin_search), la función itera a través de todos los elementos del tablero hasta que encuentra la clave buscada. La función retorna la cantidad de comparaciones realizadas durante la

búsqueda.

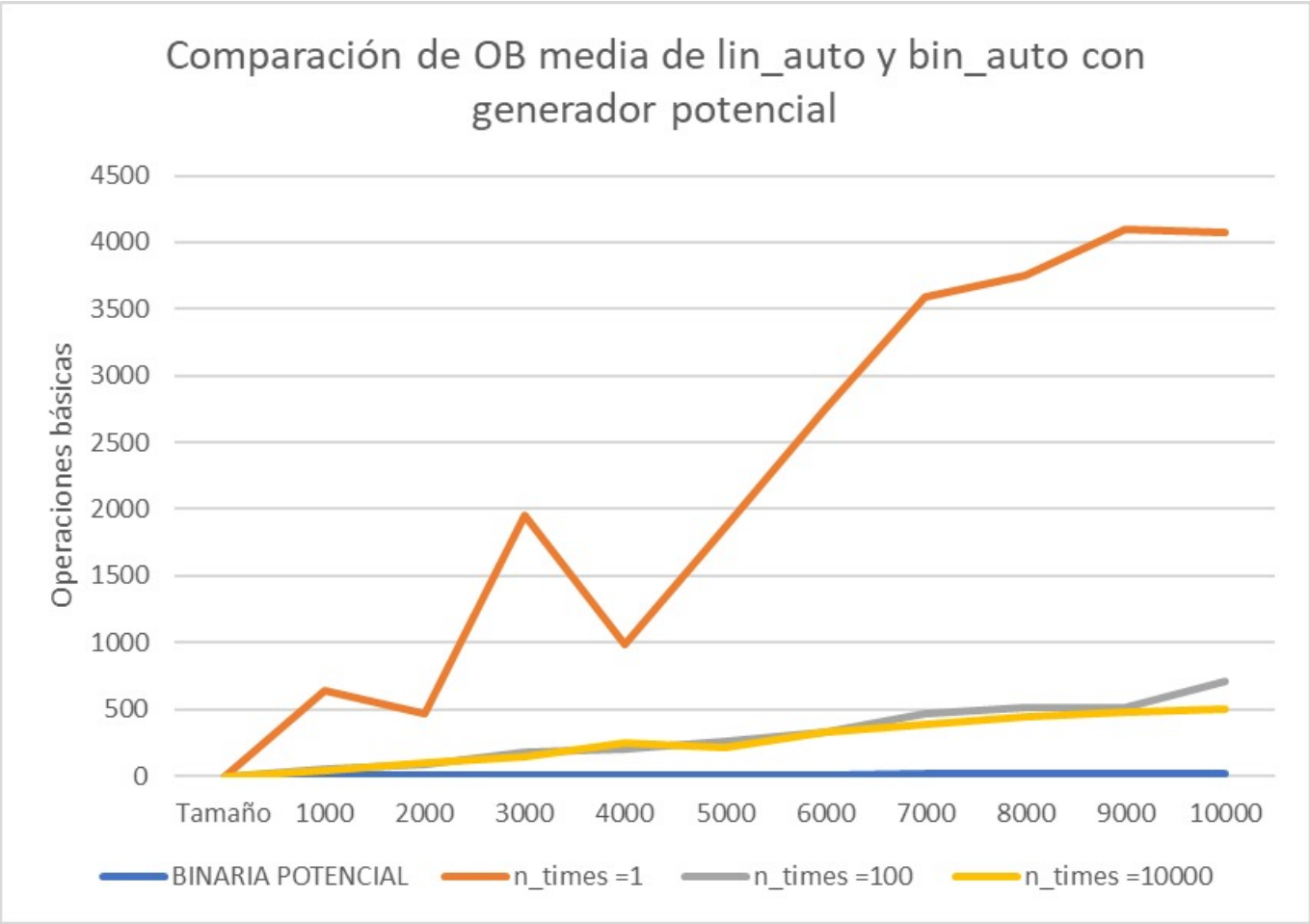
A medida que el tamaño del tablero aumenta, el tiempo promedio para la búsqueda lineal también lo hace. Esto se debe a que en el peor de los casos, la función de búsqueda lineal tiene que comparar todos los elementos del tablero antes de encontrar la clave buscada.

Por otro lado, la búsqueda binaria (`bin_search`) es un algoritmo de búsqueda más eficiente. En lugar de buscar la clave buscada a través de todos los elementos del tablero, como en la búsqueda lineal, la búsqueda binaria compara la clave buscada con el elemento del medio del tablero. Luego, si la clave buscada es igual al elemento del medio, la búsqueda se considera exitosa. Si la clave buscada es menor que el elemento del medio, la búsqueda se realiza en la mitad inferior del tablero. Si la clave buscada es mayor que el elemento del medio, la búsqueda se realiza en la mitad superior del tablero. Este proceso se repite hasta que se encuentra la clave buscada o hasta que se agota el tablero.

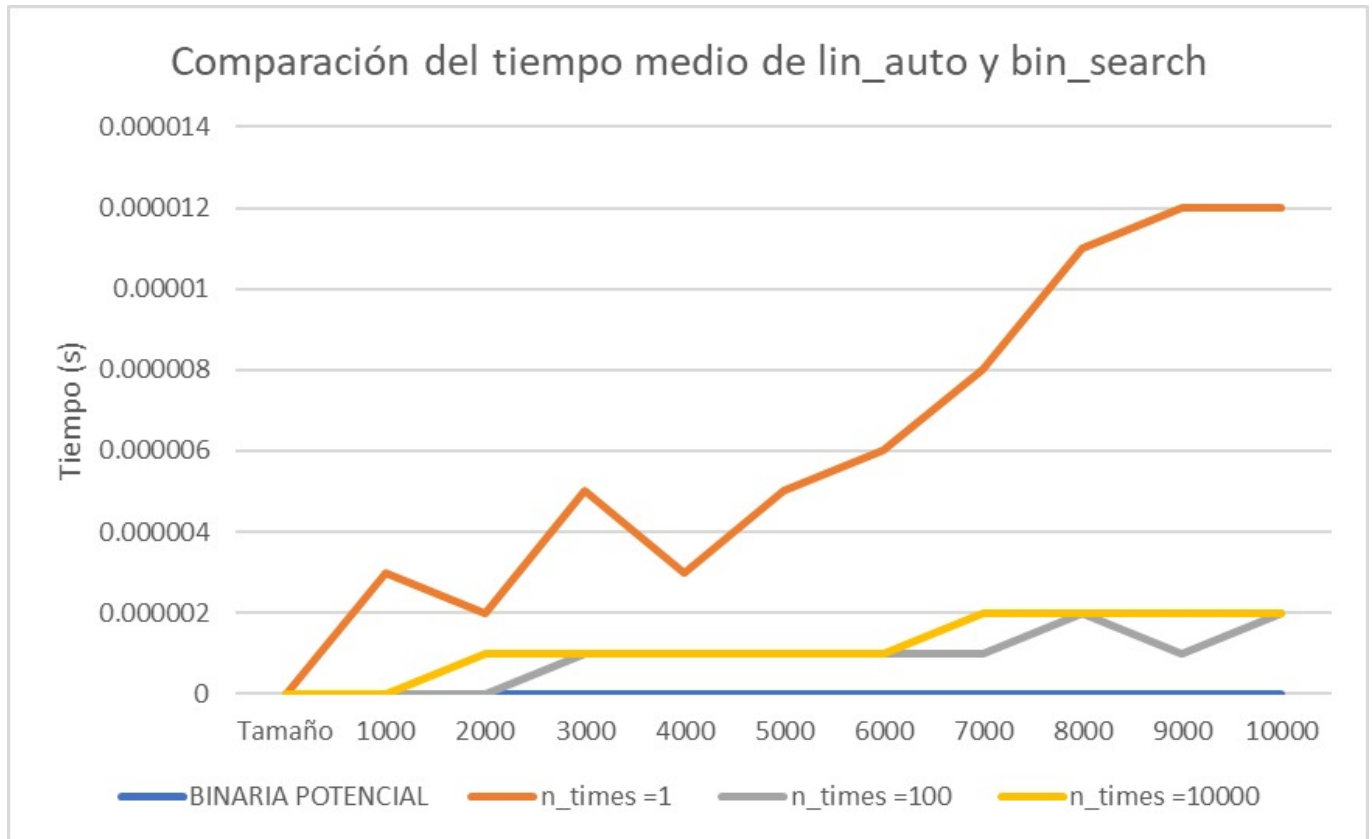
Dado que la búsqueda binaria divide el tablero en dos partes en cada paso, en el peor de los casos, la búsqueda binaria requiere un máximo de $\lceil \log(n) \rceil$ comparaciones, donde n es el tamaño del tablero.

En resumen, a medida que el tamaño del tablero aumenta, el tiempo promedio para la búsqueda binaria es significativamente menor que el tiempo promedio para la búsqueda lineal. Esto se debe a que la búsqueda binaria aprovecha la estructura del tablero para reducir el número de comparaciones necesarias en el peor de los casos.

Comando ejecutado: `./exercise2 -num_min 1000 -num_max 10000 -incr 1000 -n_times 1 -outputFile exercise5.log`



Comando ejecutado: `./exercise2 -num_min 1000 -num_max 10000 -incr 1000 -n_times 1 -outputFile exercise5.log`



La gráfica muestra las OB media (operaciones de búsqueda) para diferentes tamaños de tabla (n_times) en las funciones `lin_auto` y `bin_auto`.

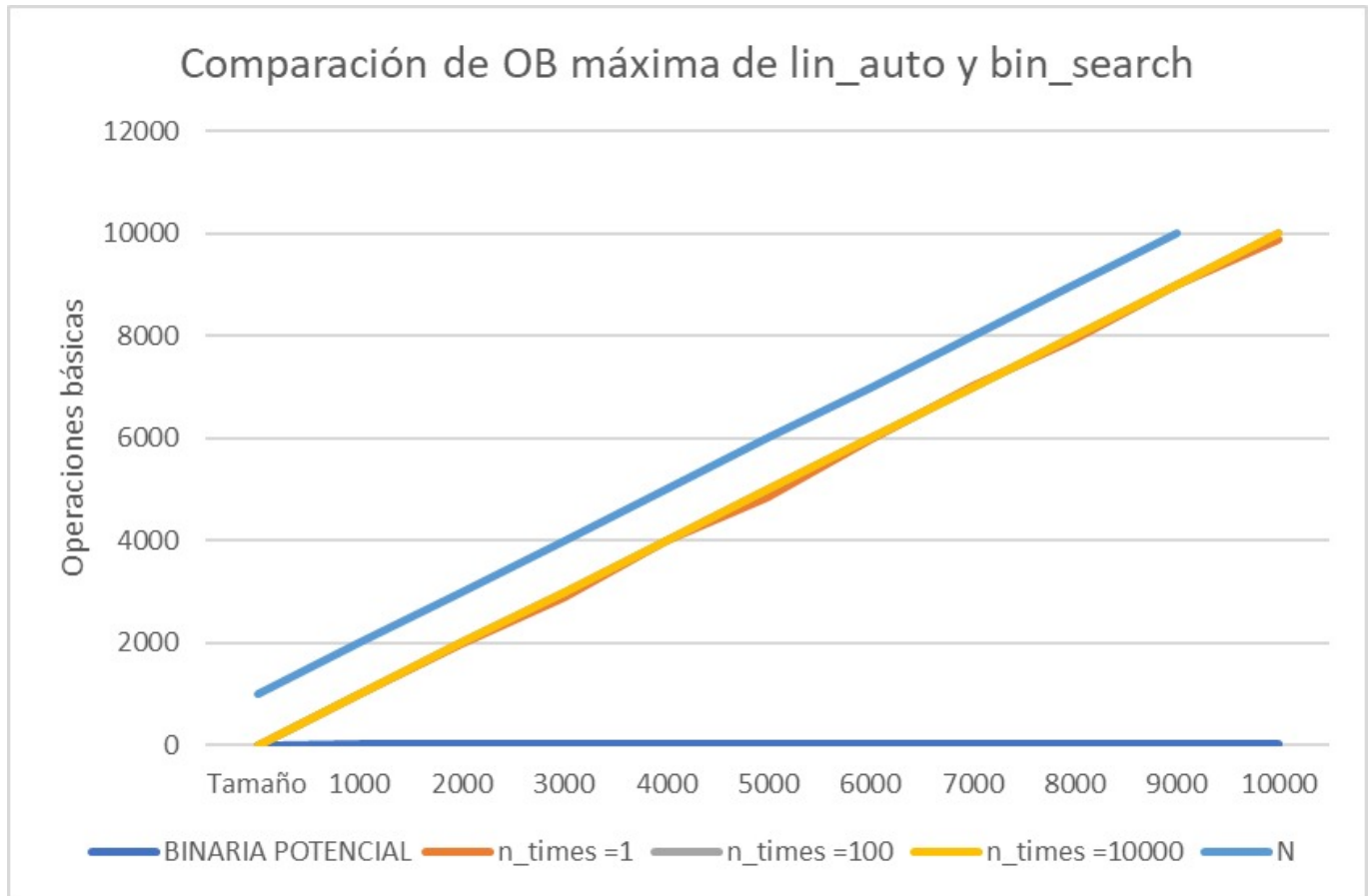
El resultado es que la búsqueda binaria (`bin_search`) tiene un tiempo promedio de operaciones de búsqueda (OB) menor que la búsqueda lineal auto organizada, que adelanta una posición la clave buscada cada vez que se encuentra, (`lin_auto`) en todas las situaciones. Esto se debe a que la búsqueda binaria utiliza una división de la tabla en cada iteración, lo que reduce significativamente el número de elementos a comparar.

La función `potencial_key_generator` es relevante para entender la naturaleza de las claves que se están utilizando en la tabla. Estas claves siguen una distribución de potencial, lo que significa que los valores más pequeños tienen una probabilidad mucho mayor de ser encontrados. Por lo tanto, la búsqueda auto organizada debería tener una ventaja sobre la búsqueda lineal en este caso, ya que la búsqueda lineal tardaría cada vez menos tiempo en encontrar los elementos más pequeños.

En resumen, la búsqueda binaria tiene un rendimiento generalmente más rápido que la búsqueda lineal, y este resultado es consistente con los diferentes tamaños de tabla y las funciones utilizadas.

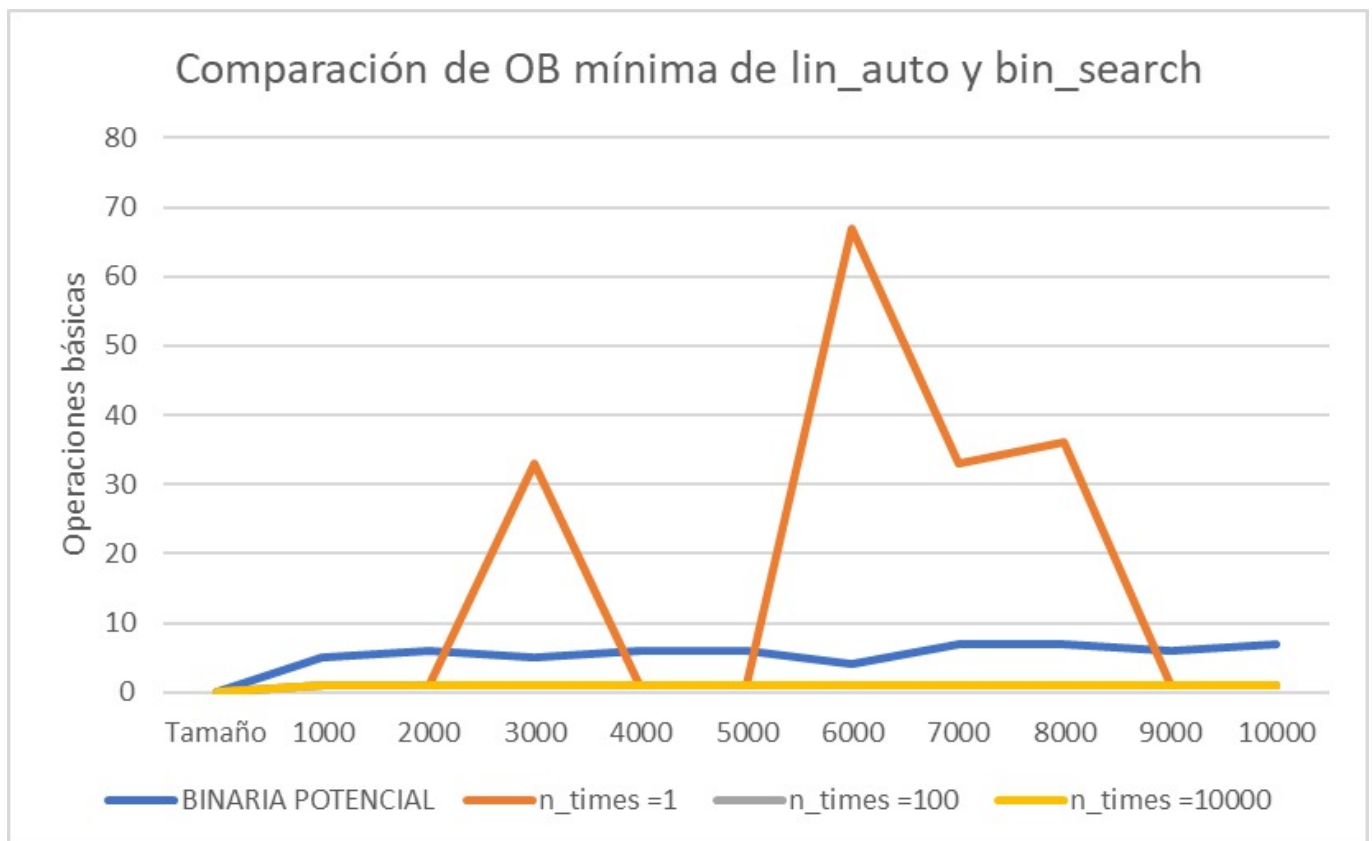
Notese que con la búsqueda binaria no se ha implementado funciones variando el n_times ya que se superponen en este contexto.

Comando ejecutado: `./exercise2 -num_min 1000 -num_max 10000 -incr 1000 -n_times 1 -outputFile exercise5.log`



Al igual que antes vemos que bin_search es extremadamente más eficiente que lin_auto en este caso lin_auto se comporta como lin_search por la propia definicion del caso peor de lin_auto.

Comando ejecutado: `./exercise2 -num_min 1000 -num_max 10000 -incr 1000 -n_times 1 -outputFile exercise5.log`



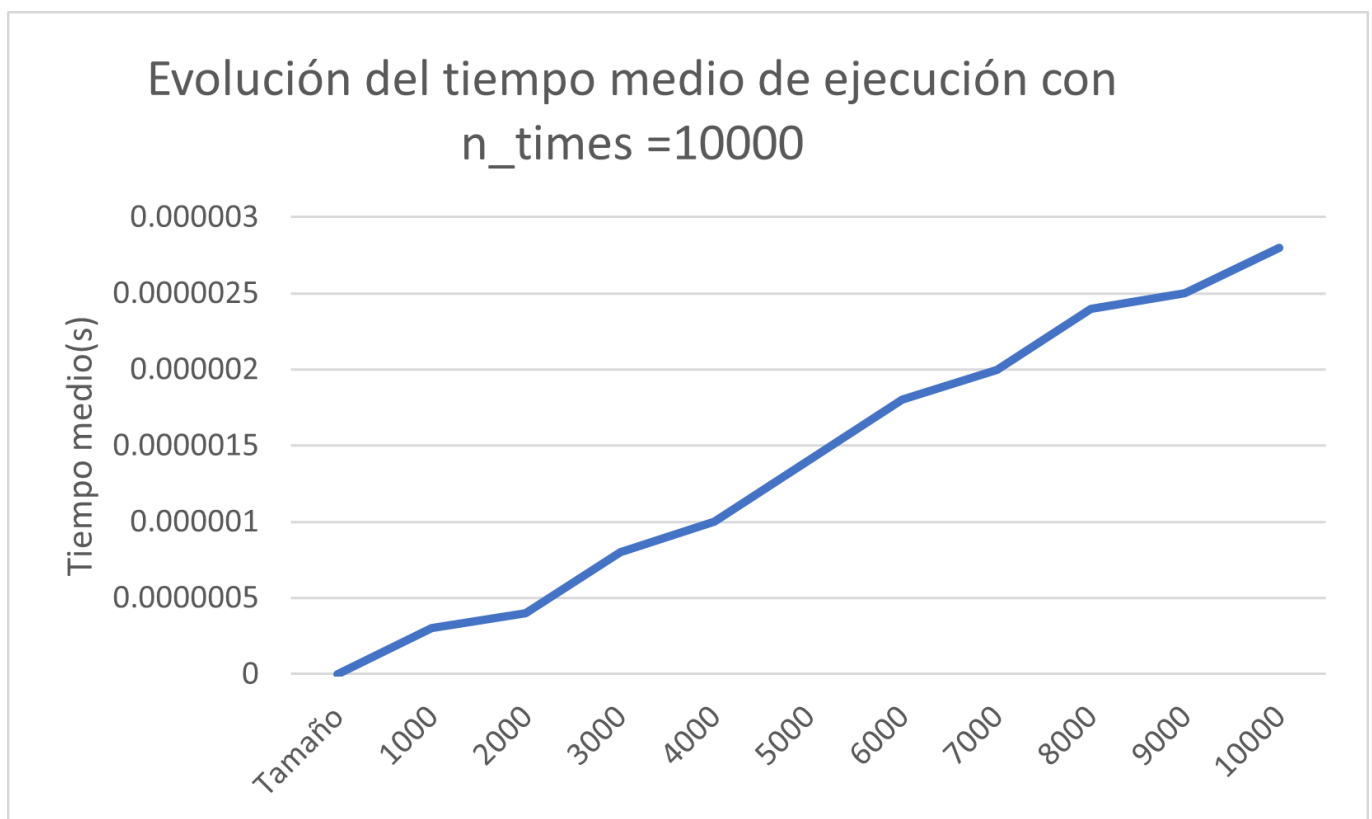
En esta gráfica comparamos las OBs minimas de bin_search y lin_auto search incrementando las veces que lin_auto busca las claves generados por potencial key generator. Como podemos ver según aumenta las veces que lin_auto busca, se reduce el numero de OBs del caso mínimo, ya que cada vez que hacemos una busqueda con este algoritmo se adelamnta una posicion el indice buscado.

Podemos ver que en n_times=100 y n_times=10000 el numero de OBs es menor que el de la busqueda binaria. Notese que las irregularidades cuando n_times=1 se deben a la dependencia del algoritmo con la poscion en la que aparecen las claves y esto sumado el reducido numero de busquedas hace que la grafica dependa de potencial key generator mientras que en el resto de casos se reduce esa dependencia al aumentar el numero de veces que se busca.

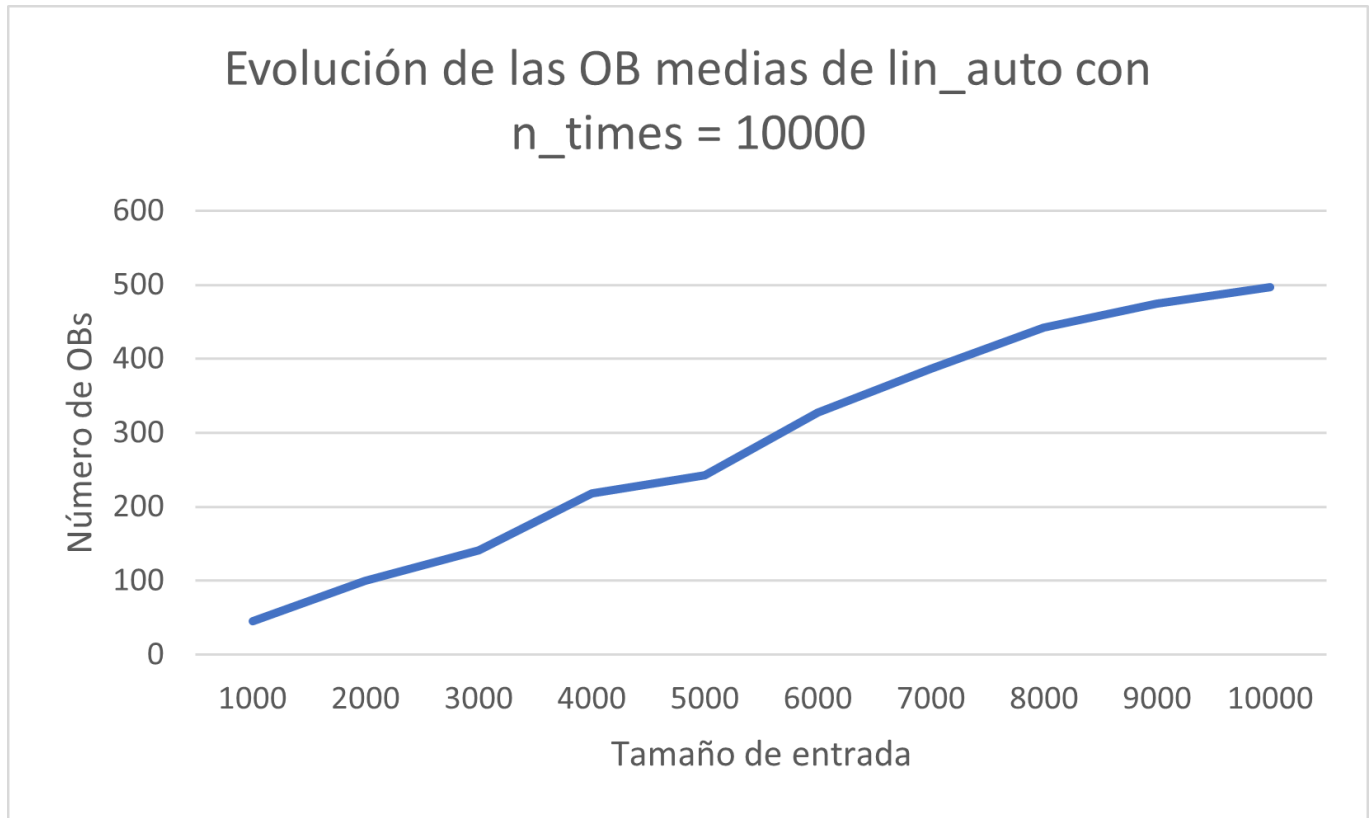
6. Respuestas a las preguntas teóricas

6.1 Pregunta 1

La operación básica de un algoritmo debe ser representativa de su funcionamiento. Además, debe ser la que más se repite, es decir, preferiblemente se elegirá una operación que se encuentre en el bucle más interno del algoritmo. Por este motivo, en lin_search la OB debe ser la comparación que se realiza en el bucle while, pues ese es el único bucle que se hace y en lin_auto_search, se debe proceder igual. Por otra parte, en bin_search también hay un solo bucle while, por lo que se elegirá de nuevo la comparación de claves como OB.



Por otra parte, bin_search tiene un caso peor $O(\log N)$. Esto se debe a que, en este caso, se está usando una estrategia "divide y vencerás". De esta forma, en cada iteración del algoritmo el tamaño de la tabla se reduce a la mitad. Esto implica que, en algún momento, si no se ha encontrado la clave, la tabla tendrá tamaño 1. No es difícil demostrar que esto lleva a $\log N$ operaciones básicas. Para forzar este caso, basta con intentar buscar una clave que se encuentre en cualquier extremo de la tabla. Por otro lado, el caso mejor sigue siendo el mismo que en la lineal, y se da cuando la clave se encuentra en la posición del medio.



6.2 Pregunta 2

Para calcular el tiempo de ejecución de los algoritmos, basta con comprobarlo de forma práctica.

Empezando con `lin_search`, claramente el mejor caso se da cuando la clave se encuentra en la primera posición de la lista, pues de esta forma solo se necesita una comparación. Esto es un rendimiento $O(1)$, y se genera si la clave está en primera posición. En cuanto al caso peor, que se da si la clave está en la última posición, o si no se encuentra en la tabla en absoluto, se necesitan tantas iteraciones como elementos tiene la tabla. En este caso, el rendimiento es $O(n)$. Con los datos empíricos se ve que se cumple la hipótesis.

6.3 Pregunta 3

`Lin_auto_search` tiene un funcionamiento similar a la búsqueda lineal normal. La única diferencia reside en que, después de encontrar la clave, se realiza un swap entre la clave buscada y el elemento anterior. De esta forma, si una clave es buscada de forma frecuente, acabará llegando a las primeras posiciones de la tabla. Así, para próximas búsquedas, las claves más comunes quedarán en las primeras posiciones y la búsqueda será más óptima. De esto, no es difícil inferir que, en un principio, tendremos un mal rendimiento (el mismo que el de `lin_search`) y, según se vayan colocando las claves llegaremos a un rendimiento bastante mejor. Esto se observa en las gráficas de apartados anteriores. Por lo tanto, según se van realizando más búsquedas, al principio de la tabla se irán acumulando las claves más frecuentes.

6.4 Pregunta 4

Para estimar el caso medio de `lin_auto_search`, hay que usar la fórmula del caso medio. Esto es:

$$A_{LA}n = \sum_{i=1}^n piOB(i)$$

En esta fórmula, $p(i)$ representa la probabilidad de que aparezca el caso i , $OB(i)$ es el número de operaciones básicas necesarias para encontrar la clave i , y n es el tamaño de la tabla.

Para calcular $p(i)$, usamos el siguiente razonamiento. La key se calcula como:

$$n = 0,5 + \frac{max}{1 + max}$$

donde x es un número aleatorio entre 0 y 1. Si despejamos la x , vemos que

$$x = \frac{1}{max} \left(\frac{max}{n - 0,5} - 1 \right)$$

Ahora, como para un tipo `int` n engloba todo el rango $[n, n+1]$, tenemos que

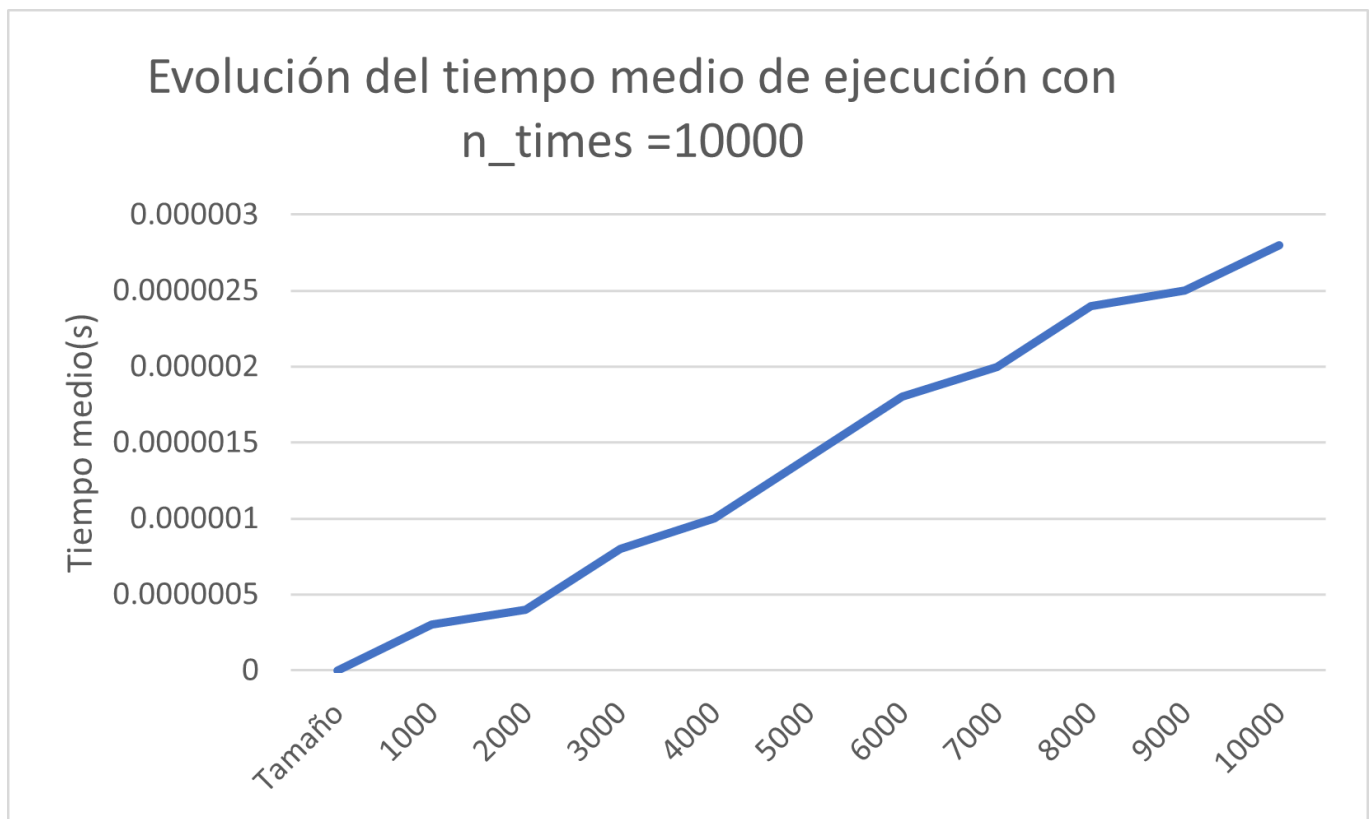
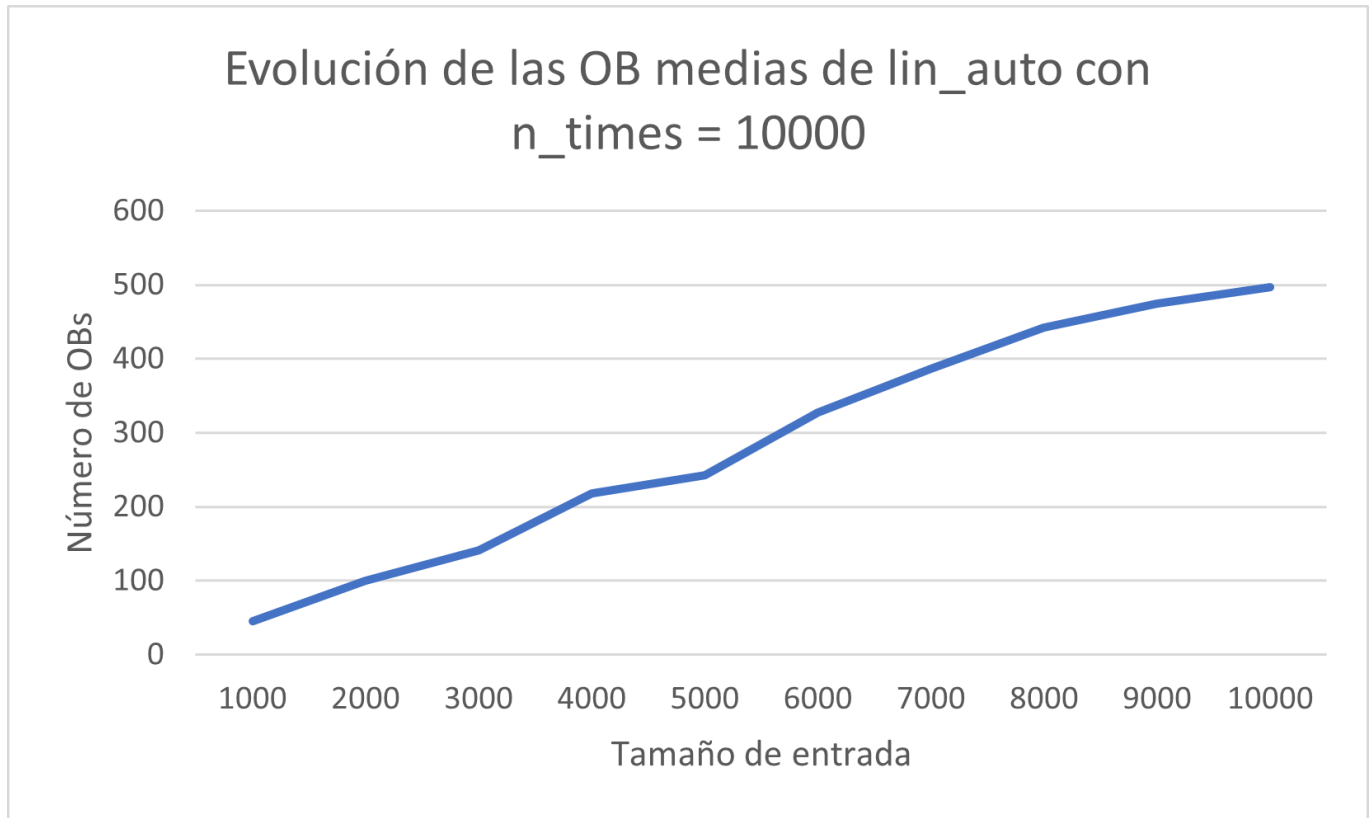
$$\frac{1}{max} \frac{max}{n - 0,5} - 1 > x \geq \frac{1}{max} \frac{max}{n + 0,5} - 1$$

Entonces, si restamos estos dos valores, obtenemos la porción de los valores entre 0 y 1 que generan n . Esto es

$$\frac{1}{max} \frac{max}{n - 0,5} - 1 - \frac{1}{max} \frac{max}{n + 0,5} - 1$$

Por otro lado, si la tabla está casi ordenada después de haber realizado muchas búsquedas, $OB_i \approx i$. Con esto, y usando la fórmula, obtenemos un crecimiento similar a $\log(n)$.

Sin embargo, en las gráficas no se observa esto, sino algo más bien lineal. Esta diferencia se puede deber a la inexactitud en este tipo de cálculos, o incluso a que no hemos tomado suficientes valores para graficar.



6.5 Pregunta 5

La clave del algoritmo `bin_search` está en el uso de la estrategia “divide y vencerás”. Esto consiste en que, en cada iteración, se reduce el problema a uno la mitad de complejo. `Lin_search`, por ejemplo, solo reduce el problema a una tabla con un elemento menos.

Esta estrategia es correcta y tiene el siguiente funcionamiento.

Se elige el elemento medio. Si este elemento resulta ser la clave, hemos terminado.

Si no, se repite el proceso con la mitad de la tabla que contiene la clave (se elige la mitad comparando el elemento medio con la clave), y se vuelve a empezar. En cierto momento, la tabla será de un único elemento y tendremos la clave o, si la clave no está en la lista, habremos terminado igualmente. Cada vez se está acotando más la solución y, como la tabla está ordenada, es imposible que se escoja la subtabla que no tiene la clave.

Esta estrategia nos lleva a un caso mejor óptimo (si la clave está en el medio de la tabla) y unos casos peor y mejor muy buenos. Podemos calcular el orden de estos casos.

El caso mejor es evidente, pues es $O(1)$ si la clave está en el medio.

Para obtener el caso peor, basta con colocar la clave, por ejemplo, en el primer espacio de la tabla. De esta forma, el número de OBs es máximo. Si la tabla tiene n elementos, hay que dividirla en 2 hasta que quede una tabla de un elemento, el buscado. De esta forma, tenemos que, en el caso peor:

$$\frac{n}{2^k} \leq 1 \implies k \leq \log(n)$$

Para el caso medio, con un razonamiento similar, se sabe que tiene una complejidad de $\log N + O(1)$.

Esta complejidad es óptima para este tipo de algoritmos. Durante esta práctica, se observa esto continuamente, pues este algoritmo es capaz de encontrar cualquier clave entre 30000 elementos en 15 iteraciones.

7.Conclusiones

En conclusión, el comportamiento del algoritmo `lin_auto_search` en la práctica no sigue perfectamente el modelo teórico propuesto. Además, factores como la aleatoriedad en la generación de claves y la naturaleza práctica de la implementación pueden influir en los resultados.

Es importante destacar que las estimaciones teóricas a veces pueden no reflejar completamente el comportamiento práctico de un algoritmo, especialmente cuando se trata de algoritmos que dependen de patrones de acceso o distribuciones específicas de datos. En este caso, la complejidad teórica puede proporcionar una guía general, pero la variabilidad de situaciones prácticas puede llevar a desviaciones de las expectativas teóricas.

Además, la implementación y el rendimiento de los algoritmos pueden estar influenciados por factores específicos de la plataforma, el compilador y otros aspectos del entorno de ejecución. Por lo tanto, es crucial realizar pruebas prácticas y análisis empíricos para comprender completamente el rendimiento de los algoritmos en situaciones del mundo real.

En conclusión, aunque las estimaciones teóricas son valiosas para comprender el comportamiento general de un algoritmo, siempre es esencial respaldarlas con pruebas prácticas y análisis empíricos para obtener una comprensión completa y precisa del rendimiento en contextos específicos.